

research

June 1, 2026

```
[1]: !cd ~/c/lu9-sql-compiler/ && ./build-release/bin/benchmarks_
      ↪--benchmark_filter='^OperatorCost/' --benchmark_out=/tmp/operator-cost.json_
      ↪--benchmark_out_format=json
```

2026-05-31T16:14:53+03:00

Running ./build-release/bin/benchmarks

Run on (8 X 4200 MHz CPU s)

CPU Caches:

L1 Data 48 KiB (x4)

L1 Instruction 32 KiB (x4)

L2 Unified 1280 KiB (x4)

L3 Unified 8192 KiB (x1)

Load Average: 4.44, 2.58, 1.61

WARNING CPU scaling is enabled, the benchmark real time measurements may be noisy and will incur extra overhead.

Benchmark			Time
CPU	Iterations	UserCounters...	

OperatorCost/SeqScan/1024/real_time

79997

ns 78049 ns 8741 model_cost=102.4k

output_rows=1.024k plan_cost=102.4k rows=1.024k

OperatorCost/SeqScan/2048/real_time

144761 ns 142224 ns 5232 model_cost=204.8k

output_rows=2.048k plan_cost=204.8k rows=2.048k

OperatorCost/SeqScan/4096/real_time

338605 ns 327024 ns 2056 model_cost=409.6k

output_rows=4.096k plan_cost=409.6k rows=4.096k

OperatorCost/SeqScan/8192/real_time

813515 ns 787366 ns 726 model_cost=819.2k

output_rows=8.192k plan_cost=819.2k rows=8.192k

OperatorCost/SeqScan/16384/real_time

2055770 ns 1963689 ns 575 model_cost=1.6384M

output_rows=16.384k plan_cost=1.6384M rows=16.384k
 OperatorCost/SeqScan/32768/real_time
 3635850 ns 3580957 ns 159 model_cost=3.2768M
 output_rows=32.768k plan_cost=3.2768M rows=32.768k
 OperatorCost/SeqScan/65536/real_time
 7848736 ns 7709842 ns 101 model_cost=6.5536M
 output_rows=65.536k plan_cost=6.5536M rows=65.536k
 OperatorCost/SeqScan/131072/real_time
 14549342 ns 14285211 ns 48 model_cost=13.1072M
 output_rows=131.072k plan_cost=13.1072M rows=131.072k
 OperatorCost/SeqScan/262144/real_time
 46805723 ns 44962657 ns 24 model_cost=26.2144M
 output_rows=262.144k plan_cost=26.2144M rows=262.144k
 OperatorCost/Filter/1024/real_time
 110848 ns 109579 ns 6051 model_cost=102.4k
 output_rows=512 plan_cost=204.8k rows=1.024k
 OperatorCost/Filter/2048/real_time
 175851 ns 174124 ns 4178 model_cost=204.8k
 output_rows=1.024k plan_cost=409.6k rows=2.048k
 OperatorCost/Filter/4096/real_time
 315093 ns 312199 ns 2166 model_cost=409.6k
 output_rows=2.048k plan_cost=819.2k rows=4.096k
 OperatorCost/Filter/8192/real_time
 673538 ns 667479 ns 1078 model_cost=819.2k
 output_rows=4.096k plan_cost=1.6384M rows=8.192k
 OperatorCost/Filter/16384/real_time
 1303801 ns 1294736 ns 456 model_cost=1.6384M
 output_rows=8.192k plan_cost=3.2768M rows=16.384k
 OperatorCost/Filter/32768/real_time
 2952031 ns 2928449 ns 256 model_cost=3.2768M
 output_rows=16.384k plan_cost=6.5536M rows=32.768k
 OperatorCost/Filter/65536/real_time
 7472369 ns 7365271 ns 102 model_cost=6.5536M
 output_rows=32.768k plan_cost=13.1072M rows=65.536k
 OperatorCost/Filter/131072/real_time
 14015431 ns 13847255 ns 58 model_cost=13.1072M
 output_rows=65.536k plan_cost=26.2144M rows=131.072k
 OperatorCost/Filter/262144/real_time
 27073700 ns 26782809 ns 24 model_cost=26.2144M
 output_rows=131.072k plan_cost=52.4288M rows=262.144k

OperatorCost/Projection/1024/real_time

131940 ns 130699 ns 5179 model_cost=22.528k
output_rows=1.024k plan_cost=124.928k rows=1.024k

OperatorCost/Projection/2048/real_time

226190 ns 225141 ns 2989 model_cost=45.056k
output_rows=2.048k plan_cost=249.856k rows=2.048k

OperatorCost/Projection/4096/real_time

514393 ns 509783 ns 1536 model_cost=90.112k
output_rows=4.096k plan_cost=499.712k rows=4.096k

OperatorCost/Projection/8192/real_time

945676 ns 940030 ns 718 model_cost=180.224k
output_rows=8.192k plan_cost=0.999424M rows=8.192k

OperatorCost/Projection/16384/real_time

1904450 ns 1892580 ns 366 model_cost=360.448k
output_rows=16.384k plan_cost=1.99885M rows=16.384k

OperatorCost/Projection/32768/real_time

3678534 ns 3656980 ns 197 model_cost=720.896k
output_rows=32.768k plan_cost=3.9977M rows=32.768k

OperatorCost/Projection/65536/real_time

7472132 ns 7428647 ns 93 model_cost=1.44179M
output_rows=65.536k plan_cost=7.99539M rows=65.536k

OperatorCost/Projection/131072/real_time

15381800 ns 15295836 ns 46 model_cost=2.88358M
output_rows=131.072k plan_cost=15.9908M rows=131.072k

OperatorCost/Projection/262144/real_time

41055491 ns 40717845 ns 19 model_cost=5.76717M
output_rows=262.144k plan_cost=31.9816M rows=262.144k

OperatorCost/Sort/1024/real_time

162085 ns 160916 ns 4340 model_cost=123.904k
output_rows=1.024k plan_cost=226.304k rows=1.024k

OperatorCost/Sort/2048/real_time

346371 ns 344873 ns 2062 model_cost=270.336k
output_rows=2.048k plan_cost=475.136k rows=2.048k

OperatorCost/Sort/4096/real_time

800255 ns 792542 ns 883 model_cost=585.728k
output_rows=4.096k plan_cost=995.328k rows=4.096k

OperatorCost/Sort/8192/real_time

1840524 ns 1830438 ns 379 model_cost=1.26157M
output_rows=8.192k plan_cost=2.08077M rows=8.192k

OperatorCost/Sort/16384/real_time

4673407 ns 4626060 ns 164 model_cost=2.70336M
output_rows=16.384k plan_cost=4.34176M rows=16.384k

OperatorCost/Sort/32768/real_time

10560526 ns 10441042 ns 60 model_cost=5.76717M
output_rows=32.768k plan_cost=9.04397M rows=32.768k

OperatorCost/Sort/65536/real_time

27512323 ns 27313637 ns 26 model_cost=12.2552M
output_rows=65.536k plan_cost=18.8088M rows=65.536k

OperatorCost/Sort/131072/real_time

76878972 ns 76256341 ns 9 model_cost=25.9523M
output_rows=131.072k plan_cost=39.0595M rows=131.072k

OperatorCost/Sort/262144/real_time

210729772 ns 208907082 ns 3 model_cost=54.7881M
output_rows=262.144k plan_cost=81.0025M rows=262.144k

OperatorCost/Aggregation/1024/real_time

367198 ns 365161 ns 2007 model_cost=522.24k
output_rows=1.024k plan_cost=624.64k rows=1.024k

OperatorCost/Aggregation/2048/real_time

714949 ns 711449 ns 1030 model_cost=1.04448M
output_rows=2.048k plan_cost=1.24928M rows=2.048k

OperatorCost/Aggregation/4096/real_time

1378878 ns 1372416 ns 506 model_cost=2.08896M
output_rows=4.096k plan_cost=2.49856M rows=4.096k

OperatorCost/Aggregation/8192/real_time

2970154 ns 2953446 ns 233 model_cost=4.17792M
output_rows=8.192k plan_cost=4.99712M rows=8.192k

OperatorCost/Aggregation/16384/real_time

7566779 ns 7487774 ns 94 model_cost=8.35584M
output_rows=16.384k plan_cost=9.99424M rows=16.384k

OperatorCost/Aggregation/32768/real_time

21294147 ns 21136412 ns 28 model_cost=16.7117M
output_rows=32.768k plan_cost=19.9885M rows=32.768k

OperatorCost/Aggregation/65536/real_time

58811825 ns 58334161 ns 11 model_cost=33.4234M
output_rows=65.536k plan_cost=39.977M rows=65.536k

OperatorCost/Aggregation/131072/real_time

145317421 ns 144445410 ns 5 model_cost=66.8467M
output_rows=131.072k plan_cost=79.9539M rows=131.072k

OperatorCost/Aggregation/262144/real_time

341980080 ns 339244361 ns 2 model_cost=133.693M
output_rows=262.144k plan_cost=159.908M rows=262.144k

OperatorCost/HashJoin/1024/1024/real_time

203598 ns 202661 ns 3434 lhs_rows=1.024k
model_cost=141.312k output_rows=1.024k plan_cost=346.112k
rhs_rows=1.024k

OperatorCost/HashJoin/2048/2048/real_time

383152 ns 381209 ns 1850 lhs_rows=2.048k
model_cost=282.624k output_rows=2.048k plan_cost=692.224k
rhs_rows=2.048k

OperatorCost/HashJoin/4096/4096/real_time

762752 ns 757843 ns 903 lhs_rows=4.096k
model_cost=565.248k output_rows=4.096k plan_cost=1.38445M
rhs_rows=4.096k

OperatorCost/HashJoin/8192/8192/real_time

1662913 ns 1651698 ns 461 lhs_rows=8.192k
model_cost=1.1305M output_rows=8.192k plan_cost=2.7689M
rhs_rows=8.192k

OperatorCost/HashJoin/16384/16384/real_time

4006295 ns 3972214 ns 191 lhs_rows=16.384k
model_cost=2.26099M output_rows=16.384k plan_cost=5.53779M
rhs_rows=16.384k

OperatorCost/HashJoin/32768/32768/real_time

8204329 ns 8095465 ns 82 lhs_rows=32.768k
model_cost=4.52198M output_rows=32.768k plan_cost=11.0756M
rhs_rows=32.768k

OperatorCost/HashJoin/65536/65536/real_time

16943075 ns 16813719 ns 43 lhs_rows=65.536k
model_cost=9.04397M output_rows=65.536k plan_cost=22.1512M
rhs_rows=65.536k

OperatorCost/NestedLoopJoin/64/64/real_time

457941 ns 454351 ns 1627 lhs_rows=64
model_cost=286.72k output_rows=64 plan_cost=299.52k rhs_rows=64

OperatorCost/NestedLoopJoin/128/128/real_time

1326260 ns 1316630 ns 462 lhs_rows=128
model_cost=1.14688M output_rows=128 plan_cost=1.17248M rhs_rows=128

OperatorCost/NestedLoopJoin/256/256/real_time

4755416 ns 4716712 ns 148 lhs_rows=256
model_cost=4.58752M output_rows=256 plan_cost=4.63872M rhs_rows=256

```

OperatorCost/NestedLoopJoin/512/512/real_time
18824481 ns      18740105 ns      39 lhs_rows=512
model_cost=18.3501M output_rows=512 plan_cost=18.4525M rhs_rows=512
OperatorCost/NestedLoopJoin/1024/1024/real_time
69821057 ns      69534977 ns      10 lhs_rows=1.024k
model_cost=73.4003M output_rows=1.024k plan_cost=73.6051M
rhs_rows=1.024k
OperatorCost/NestedLoopJoin/2048/2048/real_time
275065905 ns      274101250 ns      3 lhs_rows=2.048k
model_cost=293.601M output_rows=2.048k plan_cost=294.011M
rhs_rows=2.048k
OperatorCost/NestedLoopCrossJoin/32/32/real_time
161658 ns      160319 ns      4263 lhs_rows=32
model_cost=106.496k output_rows=1.024k plan_cost=112.896k
rhs_rows=32
OperatorCost/NestedLoopCrossJoin/64/64/real_time
391432 ns      386106 ns      1896 lhs_rows=64
model_cost=425.984k output_rows=4.096k plan_cost=438.784k
rhs_rows=64
OperatorCost/NestedLoopCrossJoin/128/128/real_time
1644326 ns      1610732 ns      331 lhs_rows=128
model_cost=1.70394M output_rows=16.384k plan_cost=1.72954M
rhs_rows=128
OperatorCost/NestedLoopCrossJoin/256/256/real_time
6994495 ns      6864530 ns      87 lhs_rows=256
model_cost=6.81574M output_rows=65.536k plan_cost=6.86694M
rhs_rows=256
OperatorCost/NestedLoopCrossJoin/512/512/real_time
27484957 ns      27127956 ns      25 lhs_rows=512
model_cost=27.263M output_rows=262.144k plan_cost=27.3654M
rhs_rows=512
OperatorCost/NestedLoopCrossJoin/1024/1024/real_time
130720901 ns      129132758 ns      5 lhs_rows=1.024k
model_cost=109.052M output_rows=1.04858M plan_cost=109.257M
rhs_rows=1.024k

```

```
[2]: !ls -l /tmp | grep operator-cost
```

```

-rw-r--r--  1 st      users    40311 May 31 16:16 operator-cost.json
-rw-r--r--  1 st      users    33223 May 31 16:03 operator-cost-matched-
calibrated-smoke.json
-rw-r--r--  1 st      users    33307 May 31 16:13 operator-cost-matched.json

```

```
-rw-r--r-- 1 st      users      6149 May 31 15:08 operator-cost-matched-plan-cost-
smoke.json
-rw-r--r-- 1 st      users      2010 May 30 19:02 operator-cost-smoke.json
```

```
[ ]: import json

results = None
with open('/tmp/operator-cost.json') as f:
    results = json.load(f)

results
```

```
[4]: import pandas as pd

assert results

df = pd.DataFrame(results['benchmarks'])
df = df[df['run_type'] == 'iteration'].copy()

parts = df['name'].str.split('/')
df['operator'] = parts.str[1]
df['left_rows'] = parts.str[2].astype(int)
df['right_rows'] = parts.apply(lambda p: int(p[3]) if len(p) == 5 else None)

df['real_time_ms'] = df['real_time'] / 1e6

df[['operator', 'left_rows', 'right_rows', 'real_time_ms', 'model_cost']].
    ↪sort_values(['operator', 'left_rows'])
```

```
[4]:
```

	operator	left_rows	right_rows	real_time_ms	model_cost
36	Aggregation	1024	NaN	0.367198	522240.0
37	Aggregation	2048	NaN	0.714949	1044480.0
38	Aggregation	4096	NaN	1.378878	2088960.0
39	Aggregation	8192	NaN	2.970154	4177920.0
40	Aggregation	16384	NaN	7.566779	8355840.0
..	...	...	...	...	...
31	Sort	16384	NaN	4.673407	2703360.0
32	Sort	32768	NaN	10.560526	5767168.0
33	Sort	65536	NaN	27.512323	12255232.0
34	Sort	131072	NaN	76.878972	25952256.0
35	Sort	262144	NaN	210.729772	54788096.0

[64 rows x 5 columns]

```
[5]: import matplotlib.pyplot as plt

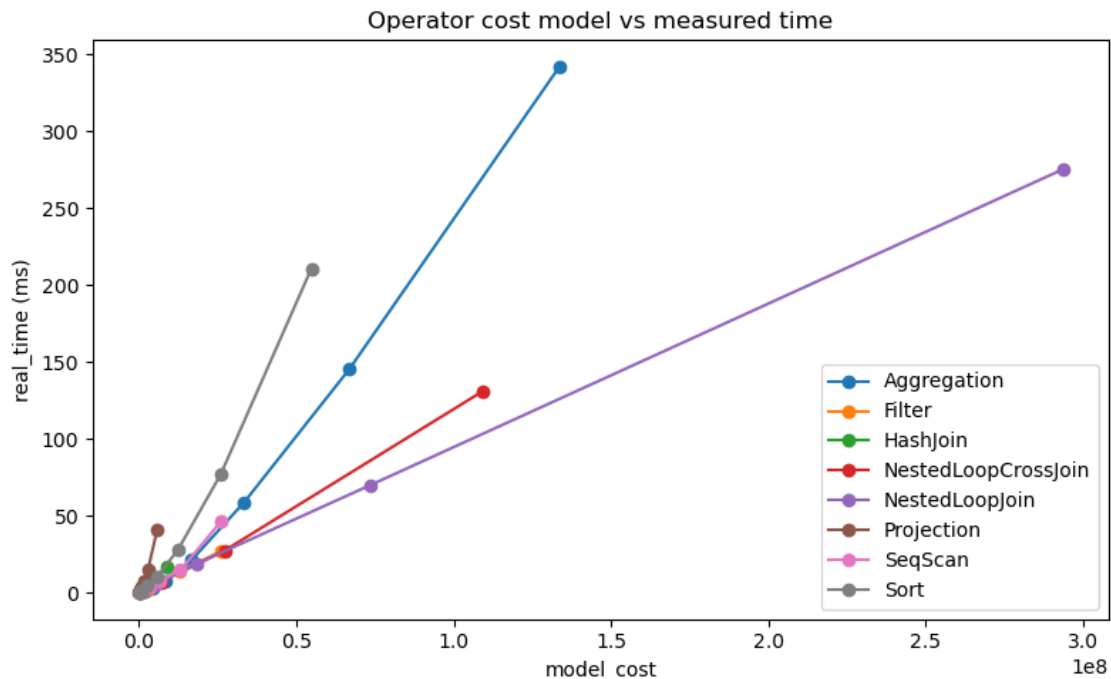
fig, ax = plt.subplots(figsize=(8, 5))
for op, grp in df.groupby('operator'):
```

```

ax.plot(grp['model_cost'], grp['real_time_ms'], marker='o', label=op)

ax.set_xlabel('model_cost')
ax.set_ylabel('real_time (ms)')
ax.set_title('Operator cost model vs measured time')
ax.legend()
plt.tight_layout()
plt.show()

```



```

[6]: import matplotlib.pyplot as plt
import numpy as np

pivot = df.set_index(['operator', 'left_rows'])[['cpu_time', 'model_cost']]

all_counts = sorted(df['left_rows'].unique())
valid_counts = [rc for rc in all_counts
                 if pivot.index.get_level_values('left_rows').isin([rc]).sum()
                 >= 2]

ncols = 2
nrows = (len(valid_counts) + ncols - 1) // ncols
fig, axes = plt.subplots(nrows, ncols, figsize=(5 * ncols, 4.5 * nrows),
                          squeeze=False)

```



```

for idx, rc in enumerate(valid_counts):
    ax = axes[idx // ncols][idx % ncols]
    sub = pivot.xs(rc, level='left_rows')
    ops = sub.index.tolist()
    m = len(ops)

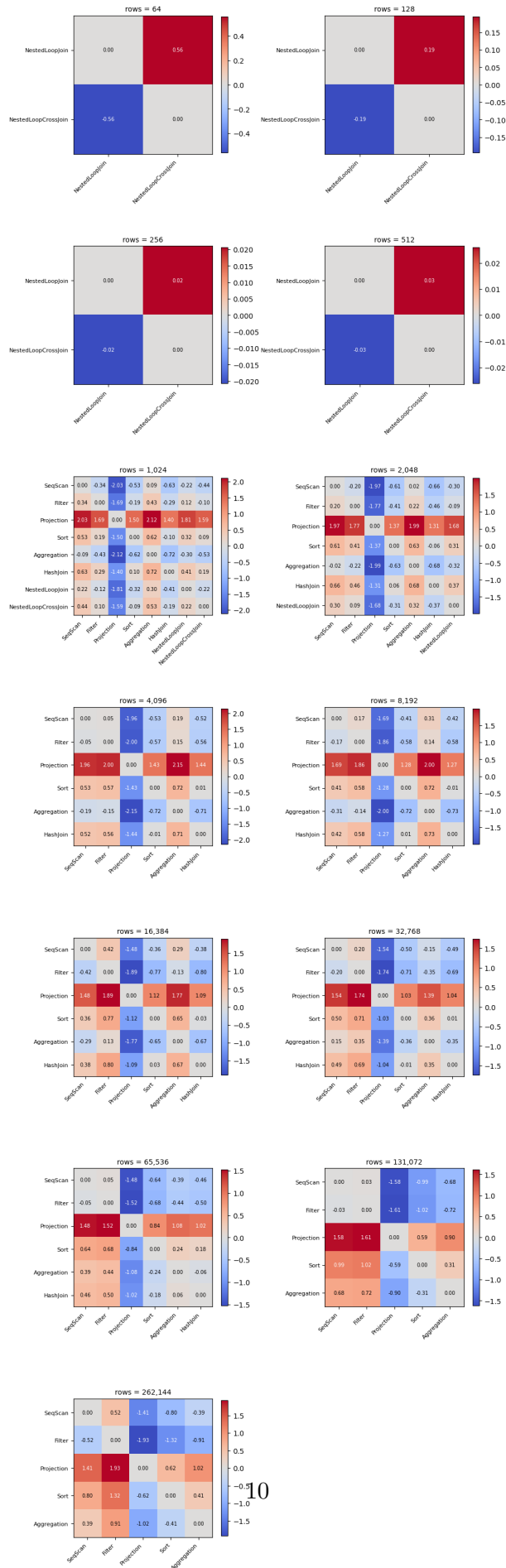
    mat = np.zeros((m, m))
    for i, a in enumerate(ops):
        for j, b in enumerate(ops):
            if i != j:
                mat[i, j] = (np.log(sub.loc[a, 'cpu_time']) - np.log(sub.
↳loc[b, 'cpu_time'])) -
                                (np.log(sub.loc[a, 'model_cost']) - np.log(sub.
↳loc[b, 'model_cost']))

    vmax = np.abs(mat).max() or 1.0
    im = ax.imshow(mat, cmap='coolwarm', vmin=-vmax, vmax=vmax)
    ax.set_xticks(range(m)); ax.set_xticklabels(ops, rotation=45, ha='right',
↳fontsize=8)
    ax.set_yticks(range(m)); ax.set_yticklabels(ops, fontsize=8)
    ax.set_title(f'rows = {rc:,}', fontsize=10)
    plt.colorbar(im, ax=ax, shrink=0.8)
    for i in range(m):
        for j in range(m):
            ax.text(j, i, f'{mat[i,j]:.2f}', ha='center', va='center',
↳fontsize=7,
                                color='black' if abs(mat[i,j]) < vmax * 0.6 else 'white')

for idx in range(len(valid_counts), nrows * ncols):
    axes[idx // ncols][idx % ncols].set_visible(False)

plt.tight_layout()
plt.show()

```



```
[7]: rows = []
for rc in valid_counts:
    sub = pivot.xs(rc, level='left_rows')
    ops = sub.index.tolist()
    for a in ops:
        for b in ops:
            if a != b:
                delta = (np.log(sub.loc[a, 'cpu_time']) - np.log(sub.loc[b,
↪'cpu_time'])) -
                    (np.log(sub.loc[a, 'model_cost']) - np.log(sub.loc[b,
↪'model_cost'])))
                rows.append({'row_count': rc, 'a': a, 'b': b, 'delta': delta})

comparison = pd.DataFrame(rows)
comparison.pivot_table(index=['a', 'b'], columns='row_count', values='delta').
↪round(3)
```

```
[7]: row_count          64      128      256      512      \
a
Aggregation      Filter      NaN      NaN      NaN      NaN
                  HashJoin     NaN      NaN      NaN      NaN
                  NestedLoopCrossJoin  NaN      NaN      NaN      NaN
                  NestedLoopJoin   NaN      NaN      NaN      NaN
                  Projection     NaN      NaN      NaN      NaN
                  SeqScan        NaN      NaN      NaN      NaN
                  Sort           NaN      NaN      NaN      NaN
Filter           Aggregation     NaN      NaN      NaN      NaN
                  HashJoin     NaN      NaN      NaN      NaN
                  NestedLoopCrossJoin  NaN      NaN      NaN      NaN
                  NestedLoopJoin   NaN      NaN      NaN      NaN
                  Projection     NaN      NaN      NaN      NaN
                  SeqScan        NaN      NaN      NaN      NaN
                  Sort           NaN      NaN      NaN      NaN
HashJoin         Aggregation     NaN      NaN      NaN      NaN
                  Filter      NaN      NaN      NaN      NaN
                  NestedLoopCrossJoin  NaN      NaN      NaN      NaN
                  NestedLoopJoin   NaN      NaN      NaN      NaN
                  Projection     NaN      NaN      NaN      NaN
                  SeqScan        NaN      NaN      NaN      NaN
                  Sort           NaN      NaN      NaN      NaN
NestedLoopCrossJoin Aggregation  NaN      NaN      NaN      NaN
                  Filter      NaN      NaN      NaN      NaN
                  HashJoin     NaN      NaN      NaN      NaN
                  NestedLoopJoin -0.559 -0.194 -0.021 -0.026
```

NestedLoopJoin	Projection	NaN	NaN	NaN	NaN	
	SeqScan	NaN	NaN	NaN	NaN	
	Sort	NaN	NaN	NaN	NaN	
	Aggregation	NaN	NaN	NaN	NaN	
	Filter	NaN	NaN	NaN	NaN	
	HashJoin	NaN	NaN	NaN	NaN	
	NestedLoopCrossJoin	0.559	0.194	0.021	0.026	
Projection	Projection	NaN	NaN	NaN	NaN	
	SeqScan	NaN	NaN	NaN	NaN	
	Sort	NaN	NaN	NaN	NaN	
	Aggregation	NaN	NaN	NaN	NaN	
	Filter	NaN	NaN	NaN	NaN	
	HashJoin	NaN	NaN	NaN	NaN	
	NestedLoopCrossJoin	NaN	NaN	NaN	NaN	
SeqScan	NestedLoopJoin	NaN	NaN	NaN	NaN	
	SeqScan	NaN	NaN	NaN	NaN	
	Sort	NaN	NaN	NaN	NaN	
	Aggregation	NaN	NaN	NaN	NaN	
	Filter	NaN	NaN	NaN	NaN	
	HashJoin	NaN	NaN	NaN	NaN	
	NestedLoopCrossJoin	NaN	NaN	NaN	NaN	
Sort	NestedLoopJoin	NaN	NaN	NaN	NaN	
	Projection	NaN	NaN	NaN	NaN	
	Sort	NaN	NaN	NaN	NaN	
	Aggregation	NaN	NaN	NaN	NaN	
	Filter	NaN	NaN	NaN	NaN	
	HashJoin	NaN	NaN	NaN	NaN	
	NestedLoopCrossJoin	NaN	NaN	NaN	NaN	
Filter	NestedLoopJoin	NaN	NaN	NaN	NaN	
	Projection	NaN	NaN	NaN	NaN	
	SeqScan	NaN	NaN	NaN	NaN	
	Sort	NaN	NaN	NaN	NaN	
	Aggregation	NaN	NaN	NaN	NaN	
	Filter	NaN	NaN	NaN	NaN	
	HashJoin	NaN	NaN	NaN	NaN	
row_count		1024	2048	4096	8192	\
a	b					
Aggregation	Filter	-0.426	-0.222	-0.149	-0.142	
	HashJoin	-0.718	-0.683	-0.713	-0.726	
	NestedLoopCrossJoin	-0.527	NaN	NaN	NaN	
	NestedLoopJoin	-0.304	-0.315	NaN	NaN	
	Projection	-2.116	-1.993	-2.153	-1.999	
	SeqScan	-0.086	-0.019	-0.195	-0.307	
	Sort	-0.619	-0.627	-0.722	-0.719	
Filter	Aggregation	0.426	0.222	0.149	0.142	
	HashJoin	-0.293	-0.461	-0.565	-0.584	
	NestedLoopCrossJoin	-0.101	NaN	NaN	NaN	
	NestedLoopJoin	0.122	-0.094	NaN	NaN	
	Projection	-1.690	-1.771	-2.004	-1.857	
	SeqScan	0.339	0.202	-0.046	-0.165	
	Sort	NaN	NaN	NaN	NaN	

HashJoin	Sort	-0.194	-0.406	-0.574	-0.577
	Aggregation	0.718	0.683	0.713	0.726
	Filter	0.293	0.461	0.565	0.584
	NestedLoopCrossJoin	0.192	NaN	NaN	NaN
	NestedLoopJoin	0.415	0.368	NaN	NaN
	Projection	-1.398	-1.310	-1.440	-1.273
	SeqScan	0.632	0.664	0.518	0.419
	Sort	0.099	0.056	-0.009	0.007
NestedLoopCrossJoin	Aggregation	0.527	NaN	NaN	NaN
	Filter	0.101	NaN	NaN	NaN
	HashJoin	-0.192	NaN	NaN	NaN
	NestedLoopJoin	0.223	NaN	NaN	NaN
	Projection	-1.589	NaN	NaN	NaN
	SeqScan	0.441	NaN	NaN	NaN
	Sort	-0.092	NaN	NaN	NaN
	Aggregation	0.304	0.315	NaN	NaN
NestedLoopJoin	Filter	-0.122	0.094	NaN	NaN
	HashJoin	-0.415	-0.368	NaN	NaN
	NestedLoopCrossJoin	-0.223	NaN	NaN	NaN
	Projection	-1.812	-1.678	NaN	NaN
	SeqScan	0.217	0.296	NaN	NaN
	Sort	-0.315	-0.312	NaN	NaN
	Aggregation	2.116	1.993	2.153	1.999
	Filter	1.690	1.771	2.004	1.857
Projection	HashJoin	1.398	1.310	1.440	1.273
	NestedLoopCrossJoin	1.589	NaN	NaN	NaN
	NestedLoopJoin	1.812	1.678	NaN	NaN
	SeqScan	2.030	1.973	1.958	1.691
	Sort	1.497	1.365	1.431	1.280
	Aggregation	0.086	0.019	0.195	0.307
	Filter	-0.339	-0.202	0.046	0.165
	HashJoin	-0.632	-0.664	-0.518	-0.419
SeqScan	NestedLoopCrossJoin	-0.441	NaN	NaN	NaN
	NestedLoopJoin	-0.217	-0.296	NaN	NaN
	Projection	-2.030	-1.973	-1.958	-1.691
	Sort	-0.533	-0.608	-0.528	-0.412
	Aggregation	0.619	0.627	0.722	0.719
	Filter	0.194	0.406	0.574	0.577
	HashJoin	-0.099	-0.056	0.009	-0.007
	NestedLoopCrossJoin	0.092	NaN	NaN	NaN
Sort	NestedLoopJoin	0.315	0.312	NaN	NaN
	Projection	-1.497	-1.365	-1.431	-1.280
	SeqScan	0.533	0.608	0.528	0.412
row_count		16384	32768	65536	131072 \
a	b				
Aggregation	Filter	0.126	0.347	0.440	0.716

Filter	HashJoin	-0.673	-0.347	-0.063	NaN
	NestedLoopCrossJoin	NaN	NaN	NaN	NaN
	NestedLoopJoin	NaN	NaN	NaN	NaN
	Projection	-1.768	-1.389	-1.083	-0.898
	SeqScan	-0.291	0.146	0.394	0.684
	Sort	-0.647	-0.359	-0.245	-0.307
	Aggregation	-0.126	-0.347	-0.440	-0.716
	HashJoin	-0.799	-0.695	-0.503	NaN
	NestedLoopCrossJoin	NaN	NaN	NaN	NaN
	NestedLoopJoin	NaN	NaN	NaN	NaN
HashJoin	Projection	-1.894	-1.736	-1.523	-1.614
	SeqScan	-0.417	-0.201	-0.046	-0.031
	Sort	-0.773	-0.706	-0.685	-1.023
	Aggregation	0.673	0.347	0.063	NaN
	Filter	0.799	0.695	0.503	NaN
	NestedLoopCrossJoin	NaN	NaN	NaN	NaN
	NestedLoopJoin	NaN	NaN	NaN	NaN
	Projection	-1.095	-1.042	-1.019	NaN
	SeqScan	0.382	0.494	0.458	NaN
	Sort	0.026	-0.011	-0.181	NaN
NestedLoopCrossJoin	Aggregation	NaN	NaN	NaN	NaN
	Filter	NaN	NaN	NaN	NaN
	HashJoin	NaN	NaN	NaN	NaN
	NestedLoopJoin	NaN	NaN	NaN	NaN
	Projection	NaN	NaN	NaN	NaN
	SeqScan	NaN	NaN	NaN	NaN
	Sort	NaN	NaN	NaN	NaN
	Aggregation	NaN	NaN	NaN	NaN
	Filter	NaN	NaN	NaN	NaN
	HashJoin	NaN	NaN	NaN	NaN
NestedLoopJoin	NestedLoopCrossJoin	NaN	NaN	NaN	NaN
	Projection	NaN	NaN	NaN	NaN
	SeqScan	NaN	NaN	NaN	NaN
	Sort	NaN	NaN	NaN	NaN
	Aggregation	NaN	NaN	NaN	NaN
	Filter	NaN	NaN	NaN	NaN
	HashJoin	NaN	NaN	NaN	NaN
	NestedLoopCrossJoin	NaN	NaN	NaN	NaN
	Projection	NaN	NaN	NaN	NaN
	SeqScan	NaN	NaN	NaN	NaN
Projection	Sort	NaN	NaN	NaN	NaN
	Aggregation	1.768	1.389	1.083	0.898
	Filter	1.894	1.736	1.523	1.614
	HashJoin	1.095	1.042	1.019	NaN
	NestedLoopCrossJoin	NaN	NaN	NaN	NaN
	NestedLoopJoin	NaN	NaN	NaN	NaN
	SeqScan	1.477	1.535	1.477	1.582
	Sort	1.121	1.030	0.838	0.591
	Aggregation	0.291	-0.146	-0.394	-0.684
	Filter	0.417	0.201	0.046	0.031
SeqScan	HashJoin	-0.382	-0.494	-0.458	NaN
	NestedLoopCrossJoin	NaN	NaN	NaN	NaN
	NestedLoopJoin	NaN	NaN	NaN	NaN
	Projection	-1.477	-1.535	-1.477	-1.582

Sort	Sort	-0.356	-0.505	-0.639	-0.992
	Aggregation	0.647	0.359	0.245	0.307
	Filter	0.773	0.706	0.685	1.023
	HashJoin	-0.026	0.011	0.181	NaN
	NestedLoopCrossJoin	NaN	NaN	NaN	NaN
	NestedLoopJoin	NaN	NaN	NaN	NaN
	Projection	-1.121	-1.030	-0.838	-0.591
	SeqScan	0.356	0.505	0.639	0.992
row_count		262144			
a	b				
Aggregation	Filter	0.910			
	HashJoin	NaN			
	NestedLoopCrossJoin	NaN			
	NestedLoopJoin	NaN			
	Projection	-1.023			
	SeqScan	0.392			
	Sort	-0.407			
	Aggregation	-0.910			
Filter	HashJoin	NaN			
	NestedLoopCrossJoin	NaN			
	NestedLoopJoin	NaN			
	Projection	-1.933			
	SeqScan	-0.518			
	Sort	-1.317			
	Aggregation	NaN			
	Filter	NaN			
HashJoin	NestedLoopCrossJoin	NaN			
	NestedLoopJoin	NaN			
	Projection	NaN			
	SeqScan	NaN			
	Sort	NaN			
	Aggregation	NaN			
	Filter	NaN			
	NestedLoopJoin	NaN			
NestedLoopCrossJoin	Projection	NaN			
	SeqScan	NaN			
	Sort	NaN			
	Aggregation	NaN			
	Filter	NaN			
	HashJoin	NaN			
	NestedLoopJoin	NaN			
	Projection	NaN			
NestedLoopJoin	SeqScan	NaN			
	Sort	NaN			
	Aggregation	NaN			
	Filter	NaN			
	HashJoin	NaN			
	NestedLoopCrossJoin	NaN			
	Projection	NaN			
	SeqScan	NaN			
Projection	Sort	NaN			
	Aggregation	1.023			

	Filter	1.933
	HashJoin	NaN
	NestedLoopCrossJoin	NaN
	NestedLoopJoin	NaN
	SeqScan	1.415
	Sort	0.616
SeqScan	Aggregation	-0.392
	Filter	0.518
	HashJoin	NaN
	NestedLoopCrossJoin	NaN
	NestedLoopJoin	NaN
	Projection	-1.415
	Sort	-0.799
Sort	Aggregation	0.407
	Filter	1.317
	HashJoin	NaN
	NestedLoopCrossJoin	NaN
	NestedLoopJoin	NaN
	Projection	-0.616
	SeqScan	0.799

```
[8]: !cd ~/c/lu9-sql-compiler/ && ./build-release/bin/benchmarks_
      ↪--benchmark_filter='OperatorCostMatched*' --benchmark_out=/tmp/
      ↪operator-cost-matched.json --benchmark_out_format=json
```

2026-05-31T16:16:05+03:00

Running ./build-release/bin/benchmarks

Run on (8 X 4200 MHz CPU s)

CPU Caches:

L1 Data 48 KiB (x4)

L1 Instruction 32 KiB (x4)

L2 Unified 1280 KiB (x4)

L3 Unified 8192 KiB (x1)

Load Average: 3.38, 2.81, 1.77

WARNING CPU scaling is enabled, the benchmark real time measurements may be noisy and will incur extra overhead.

Benchmark

Time CPU Iterations UserCounters...

OperatorCostMatched/target_cost:640000/SeqScan/6400/real_time

532418 ns 521056 ns 1116

model_cost=640k output_rows=6.4k plan_cost=640k rows=6.4k

OperatorCostMatched/target_cost:640000/Filter/6400/real_time

508270 ns 501445 ns 1089

model_cost=640k output_rows=3.328k plan_cost=1.28M rows=6.4k
 OperatorCostMatched/target_cost:640000/Projection/29091/real_time
 4068856 ns 4010747 ns 156
 model_cost=640.002k output_rows=29.091k plan_cost=3.5491M
 rows=29.091k
 OperatorCostMatched/target_cost:640000/Sort/4476/real_time
 997566 ns 988471 ns 786
 model_cost=640.068k output_rows=4.476k plan_cost=1.08767M
 rows=4.476k
 OperatorCostMatched/target_cost:640000/Aggregation/1255/real_time
 529187 ns 520615 ns 1000
 model_cost=640.05k output_rows=1.255k plan_cost=765.55k rows=1.255k
 OperatorCostMatched/target_cost:640000/HashJoin/4638/4638/real_time
 1026885 ns 1016820 ns 714
 lhs_rows=4.638k model_cost=640.044k output_rows=4.638k
 plan_cost=1.56764M rhs_rows=4.638k
 OperatorCostMatched/target_cost:640000/NestedLoopJoin/96/96/real_time
 875389 ns 866351 ns 804 lhs_rows=96
 model_cost=645.12k output_rows=96 plan_cost=664.32k rhs_rows=96
 OperatorCostMatched/target_cost:640000/NestedLoopCrossJoin/78/78/real_time
 time 527338 ns 522288 ns 1107
 lhs_rows=78 model_cost=632.736k output_rows=6.084k
 plan_cost=648.336k rhs_rows=78
 OperatorCostMatched/target_cost:1000000/SeqScan/10000/real_time
 507202 ns 502389 ns 1000
 model_cost=1M output_rows=10k plan_cost=1M rows=10k
 OperatorCostMatched/target_cost:1000000/Filter/10000/real_time
 861315 ns 854085 ns 818
 model_cost=1M output_rows=5.12k plan_cost=2M rows=10k
 OperatorCostMatched/target_cost:1000000/Projection/45455/real_time
 6698145 ns 6632289 ns 94
 model_cost=1.00001M output_rows=45.455k plan_cost=5.54551M
 rows=45.455k
 OperatorCostMatched/target_cost:1000000/Sort/6993/real_time
 1503435 ns 1491454 ns 442
 model_cost=0.999999M output_rows=6.993k plan_cost=1.6993M
 rows=6.993k
 OperatorCostMatched/target_cost:1000000/Aggregation/1961/real_time
 702024 ns 697631 ns 968
 model_cost=1.00011M output_rows=1.961k plan_cost=1.19621M
 rows=1.961k

OperatorCostMatched/target_cost:1000000/HashJoin/7246/7246/real_time
1406982 ns 1398061 ns 445
lhs_rows=7.246k model_cost=0.999948M output_rows=7.246k
plan_cost=2.44915M rhs_rows=7.246k
OperatorCostMatched/target_cost:1000000/NestedLoopJoin/120/120/real_time
me 1204952 ns 1197240 ns 548
lhs_rows=120 model_cost=1.008M output_rows=120 plan_cost=1.032M
rhs_rows=120
OperatorCostMatched/target_cost:1000000/NestedLoopCrossJoin/98/98/real_time
735577 ns 729661 ns 851
lhs_rows=98 model_cost=998.816k output_rows=9.604k
plan_cost=1.01842M rhs_rows=98
OperatorCostMatched/target_cost:3240000/SeqScan/32400/real_time
1552618 ns 1542405 ns 398
model_cost=3.24M output_rows=32.4k plan_cost=3.24M rows=32.4k
OperatorCostMatched/target_cost:3240000/Filter/32400/real_time
2615187 ns 2600696 ns 253
model_cost=3.24M output_rows=16.384k plan_cost=6.48M rows=32.4k
OperatorCostMatched/target_cost:3240000/Projection/147273/real_time
24615070 ns 24428312 ns 28
model_cost=3.24001M output_rows=147.273k plan_cost=17.9673M
rows=147.273k
OperatorCostMatched/target_cost:3240000/Sort/19636/real_time
5716537 ns 5678245 ns 122
model_cost=3.23994M output_rows=19.636k plan_cost=5.20354M
rows=19.636k
OperatorCostMatched/target_cost:3240000/Aggregation/6353/real_time
2528047 ns 2507421 ns 253
model_cost=3.24003M output_rows=6.353k plan_cost=3.87533M
rows=6.353k
OperatorCostMatched/target_cost:3240000/HashJoin/23478/23478/real_time
5300551 ns 5262456 ns 116
lhs_rows=23.478k model_cost=3.23996M output_rows=23.478k
plan_cost=7.93556M rhs_rows=23.478k
OperatorCostMatched/target_cost:3240000/NestedLoopJoin/215/215/real_time
me 3440769 ns 3423785 ns 199
lhs_rows=215 model_cost=3.23575M output_rows=215 plan_cost=3.27875M
rhs_rows=215
OperatorCostMatched/target_cost:3240000/NestedLoopCrossJoin/177/177/real_time
2294554 ns 2274793 ns 241
lhs_rows=177 model_cost=3.25822M output_rows=31.329k

```

plan_cost=3.29362M rhs_rows=177
OperatorCostMatched/target_cost:6760000/SeqScan/67600/real_time
    4220053 ns    4185107 ns    169
model_cost=6.76M output_rows=67.6k plan_cost=6.76M rows=67.6k
OperatorCostMatched/target_cost:6760000/Filter/67600/real_time
    5592034 ns    5560841 ns    126
model_cost=6.76M output_rows=33.808k plan_cost=13.52M rows=67.6k
OperatorCostMatched/target_cost:6760000/Projection/307273/real_time
    51498743 ns    51127846 ns    14
model_cost=6.76001M output_rows=307.273k plan_cost=37.4873M
rows=307.273k
OperatorCostMatched/target_cost:6760000/Sort/38409/real_time
    12802465 ns    12732765 ns    50
model_cost=6.75998M output_rows=38.409k plan_cost=10.6009M
rows=38.409k
OperatorCostMatched/target_cost:6760000/Aggregation/13255/real_time
    5687200 ns    5655535 ns    118
model_cost=6.76005M output_rows=13.255k plan_cost=8.08555M
rows=13.255k
OperatorCostMatched/target_cost:6760000/HashJoin/48986/48986/real_time
    13564179 ns    13440945 ns    51
lhs_rows=48.986k model_cost=6.76007M output_rows=48.986k
plan_cost=16.5573M rhs_rows=48.986k
OperatorCostMatched/target_cost:6760000/NestedLoopJoin/311/311/real_time
me    7198310 ns    7163077 ns    93
lhs_rows=311 model_cost=6.77047M output_rows=311 plan_cost=6.83267M
rhs_rows=311
OperatorCostMatched/target_cost:6760000/NestedLoopCrossJoin/255/255/real_time
    6012240 ns    5936779 ns    87
lhs_rows=255 model_cost=6.7626M output_rows=65.025k
plan_cost=6.8136M rhs_rows=255
OperatorCostMatched/target_cost:12250000/SeqScan/122500/real_time
    7610143 ns    7525813 ns    83
model_cost=12.25M output_rows=122.5k plan_cost=12.25M rows=122.5k
OperatorCostMatched/target_cost:12250000/Filter/122500/real_time
    10313976 ns    10227032 ns    63
model_cost=12.25M output_rows=61.44k plan_cost=24.5M rows=122.5k
OperatorCostMatched/target_cost:12250000/Projection/556818/real_time
    90203371 ns    89533850 ns    8
model_cost=12.25M output_rows=556.818k plan_cost=67.9318M
rows=556.818k

```

OperatorCostMatched/target_cost:12250000/Sort/65536/real_time
26495662 ns 26260681 ns 22
model_cost=12.2552M output_rows=65.536k plan_cost=18.8088M
rows=65.536k
OperatorCostMatched/target_cost:12250000/Aggregation/24020/real_time
15047581 ns 14860662 ns 50
model_cost=12.2502M output_rows=24.02k plan_cost=14.6522M
rows=24.02k
OperatorCostMatched/target_cost:12250000/HashJoin/88768/88768/real_time
e 25706484 ns 25401595 ns 28
lhs_rows=88.768k model_cost=12.25M output_rows=88.768k
plan_cost=30.0036M rhs_rows=88.768k
OperatorCostMatched/target_cost:12250000/NestedLoopJoin/418/418/real_time
ime 12686905 ns 12628552 ns 47
lhs_rows=418 model_cost=12.2307M output_rows=418 plan_cost=12.3143M
rhs_rows=418
OperatorCostMatched/target_cost:12250000/NestedLoopCrossJoin/343/343/real_time
eal_time 12634892 ns 12474012 ns 52
lhs_rows=343 model_cost=12.2355M output_rows=117.649k
plan_cost=12.3041M rhs_rows=343
OperatorCostMatched/target_cost:26010000/SeqScan/260100/real_time
18734465 ns 18479865 ns 37
model_cost=26.01M output_rows=260.1k plan_cost=26.01M rows=260.1k
OperatorCostMatched/target_cost:26010000/Filter/260100/real_time
26934394 ns 26755899 ns 28
model_cost=26.01M output_rows=130.052k plan_cost=52.02M rows=260.1k
OperatorCostMatched/target_cost:26010000/Projection/1182273/real_time
202987386 ns 201290217 ns 4
model_cost=26.01M output_rows=1.18227M plan_cost=144.237M
rows=1.18227M
OperatorCostMatched/target_cost:26010000/Sort/131364/real_time
74202993 ns 73103763 ns 9
model_cost=26.0101M output_rows=131.364k plan_cost=39.1465M
rows=131.364k
OperatorCostMatched/target_cost:26010000/Aggregation/51000/real_time
48595095 ns 48139275 ns 16
model_cost=26.01M output_rows=51k plan_cost=31.11M rows=51k
OperatorCostMatched/target_cost:26010000/HashJoin/188478/188478/real_time
ime 52584297 ns 52158890 ns 14
lhs_rows=188.478k model_cost=26.01M output_rows=188.478k
plan_cost=63.7056M rhs_rows=188.478k

OperatorCostMatched/target_cost:26010000/NestedLoopJoin/610/610/real_t

ime 25448562 ns 25343677 ns 26

lhs_rows=610 model_cost=26.047M output_rows=610 plan_cost=26.169M

rhs_rows=610

OperatorCostMatched/target_cost:26010000/NestedLoopCrossJoin/500/500/r

eal_time 30993358 ns 30600644 ns 27

lhs_rows=500 model_cost=26M output_rows=250k plan_cost=26.1M

rhs_rows=500

```
[ ]: import json
results = None
with open('/tmp/operator-cost-matched.json') as f:
    results = json.load(f)

results
```

```
[10]: matched_df = pd.DataFrame(results['benchmarks'])
matched_df = matched_df[matched_df['run_type'] == 'iteration'].copy()

parts = matched_df['name'].str.split('/')
matched_df['target_cost'] = parts.str[1].str.removeprefix('target_cost:').
    ↪astype(int)
matched_df['operator'] = parts.str[2]
matched_df['left_rows'] = parts.str[3].astype(int)
matched_df['right_rows'] = parts.apply(lambda p: int(p[4]) if len(p) == 6 else
    ↪None)
matched_df['real_time_ms'] = matched_df['real_time'] / 1e6

matched_df['matching_error'] = (matched_df['model_cost'] -
    ↪matched_df['target_cost']).abs() / matched_df['target_cost']
assert (matched_df['matching_error'] < 0.02).all()

matched_df[['target_cost', 'operator', 'left_rows', 'right_rows',
    ↪'real_time_ms', 'model_cost', 'plan_cost', 'matching_error']].
    ↪sort_values(['target_cost', 'operator'])
```

```
[10]:
```

	target_cost	operator	left_rows	right_rows	real_time_ms	\
4	640000	Aggregation	1255	NaN	0.529187	
1	640000	Filter	6400	NaN	0.508270	
5	640000	HashJoin	4638	4638.0	1.026885	
7	640000	NestedLoopCrossJoin	78	78.0	0.527338	
6	640000	NestedLoopJoin	96	96.0	0.875389	
2	640000	Projection	29091	NaN	4.068856	
0	640000	SeqScan	6400	NaN	0.532418	
3	640000	Sort	4476	NaN	0.997566	

12	1000000	Aggregation	1961	NaN	0.702024
9	1000000	Filter	10000	NaN	0.861315
13	1000000	HashJoin	7246	7246.0	1.406982
15	1000000	NestedLoopCrossJoin	98	98.0	0.735577
14	1000000	NestedLoopJoin	120	120.0	1.204952
10	1000000	Projection	45455	NaN	6.698145
8	1000000	SeqScan	10000	NaN	0.507202
11	1000000	Sort	6993	NaN	1.503435
20	3240000	Aggregation	6353	NaN	2.528047
17	3240000	Filter	32400	NaN	2.615187
21	3240000	HashJoin	23478	23478.0	5.300551
23	3240000	NestedLoopCrossJoin	177	177.0	2.294554
22	3240000	NestedLoopJoin	215	215.0	3.440769
18	3240000	Projection	147273	NaN	24.615070
16	3240000	SeqScan	32400	NaN	1.552618
19	3240000	Sort	19636	NaN	5.716537
28	6760000	Aggregation	13255	NaN	5.687200
25	6760000	Filter	67600	NaN	5.592034
29	6760000	HashJoin	48986	48986.0	13.564179
31	6760000	NestedLoopCrossJoin	255	255.0	6.012240
30	6760000	NestedLoopJoin	311	311.0	7.198310
26	6760000	Projection	307273	NaN	51.498743
24	6760000	SeqScan	67600	NaN	4.220053
27	6760000	Sort	38409	NaN	12.802465
36	12250000	Aggregation	24020	NaN	15.047581
33	12250000	Filter	122500	NaN	10.313976
37	12250000	HashJoin	88768	88768.0	25.706484
39	12250000	NestedLoopCrossJoin	343	343.0	12.634892
38	12250000	NestedLoopJoin	418	418.0	12.686905
34	12250000	Projection	556818	NaN	90.203371
32	12250000	SeqScan	122500	NaN	7.610143
35	12250000	Sort	65536	NaN	26.495662
44	26010000	Aggregation	51000	NaN	48.595095
41	26010000	Filter	260100	NaN	26.934394
45	26010000	HashJoin	188478	188478.0	52.584297
47	26010000	NestedLoopCrossJoin	500	500.0	30.993358
46	26010000	NestedLoopJoin	610	610.0	25.448562
42	26010000	Projection	1182273	NaN	202.987386
40	26010000	SeqScan	260100	NaN	18.734465
43	26010000	Sort	131364	NaN	74.202993

	model_cost	plan_cost	matching_error
4	640050.0	765550.0	7.812500e-05
1	640000.0	1280000.0	0.000000e+00
5	640044.0	1567644.0	6.875000e-05
7	632736.0	648336.0	1.135000e-02
6	645120.0	664320.0	8.000000e-03

2	640002.0	3549102.0	3.125000e-06
0	640000.0	640000.0	0.000000e+00
3	640068.0	1087668.0	1.062500e-04
12	1000110.0	1196210.0	1.100000e-04
9	1000000.0	2000000.0	0.000000e+00
13	999948.0	2449148.0	5.200000e-05
15	998816.0	1018416.0	1.184000e-03
14	1008000.0	1032000.0	8.000000e-03
10	1000010.0	5545510.0	1.000000e-05
8	1000000.0	1000000.0	0.000000e+00
11	999999.0	1699299.0	1.000000e-06
20	3240030.0	3875330.0	9.259259e-06
17	3240000.0	6480000.0	0.000000e+00
21	3239964.0	7935564.0	1.111111e-05
23	3258216.0	3293616.0	5.622222e-03
22	3235750.0	3278750.0	1.311728e-03
18	3240006.0	17967306.0	1.851852e-06
16	3240000.0	3240000.0	0.000000e+00
19	3239940.0	5203540.0	1.851852e-05
28	6760050.0	8085550.0	7.396450e-06
25	6760000.0	13520000.0	0.000000e+00
29	6760068.0	16557268.0	1.005917e-05
31	6762600.0	6813600.0	3.846154e-04
30	6770470.0	6832670.0	1.548817e-03
26	6760006.0	37487306.0	8.875740e-07
24	6760000.0	6760000.0	0.000000e+00
27	6759984.0	10600884.0	2.366864e-06
36	12250200.0	14652200.0	1.632653e-05
33	12250000.0	24500000.0	0.000000e+00
37	12249984.0	30003584.0	1.306122e-06
39	12235496.0	12304096.0	1.184000e-03
38	12230680.0	12314280.0	1.577143e-03
34	12249996.0	67931796.0	3.265306e-07
32	12250000.0	12250000.0	0.000000e+00
35	12255232.0	18808832.0	4.271020e-04
44	26010000.0	31110000.0	0.000000e+00
41	26010000.0	52020000.0	0.000000e+00
45	26009964.0	63705564.0	1.384083e-06
47	26000000.0	26100000.0	3.844675e-04
46	26047000.0	26169000.0	1.422530e-03
42	26010006.0	144237306.0	2.306805e-07
40	26010000.0	26010000.0	0.000000e+00
43	26010072.0	39146472.0	2.768166e-06

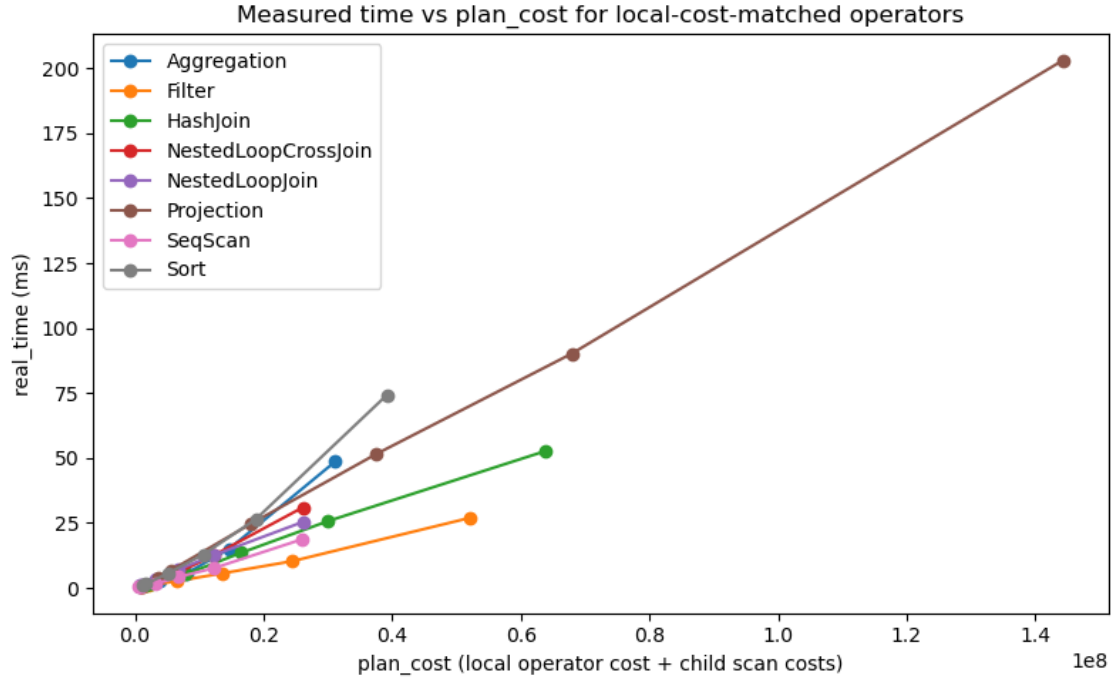
```
[11]: fig, ax = plt.subplots(figsize=(8, 5))
      for op, grp in matched_df.groupby('operator'):
          grp = grp.sort_values('plan_cost')
```

```

ax.plot(grp['plan_cost'], grp['real_time_ms'], marker='o', label=op)

ax.set_xlabel('plan_cost (local operator cost + child scan costs)')
ax.set_ylabel('real_time (ms)')
ax.set_title('Measured time vs plan_cost for local-cost-matched operators')
ax.legend()
plt.tight_layout()
plt.show()

```



```

[12]: matched_pivot = matched_df.set_index(['target_cost', 'operator'])[['cpu_time', '
    ↳ 'model_cost', 'plan_cost']]
matched_costs = sorted(matched_df['target_cost'].unique())

ncols = 2
nrows = (len(matched_costs) + ncols - 1) // ncols
fig, axes = plt.subplots(nrows, ncols, figsize=(5 * ncols, 4.5 * nrows),
    ↳ squeeze=False)

for idx, target_cost in enumerate(matched_costs):
    ax = axes[idx // ncols][idx % ncols]
    sub = matched_pivot.xs(target_cost, level='target_cost')
    ops = sub.index.tolist()
    m = len(ops)

```



```

mat = np.zeros((m, m))
for i, a in enumerate(ops):
    for j, b in enumerate(ops):
        if i != j:
            mat[i, j] = (np.log(sub.loc[a, 'cpu_time']) - np.log(sub.loc[b,
↪'cpu_time']) -
                                (np.log(sub.loc[a, 'plan_cost']) - np.log(sub.
↪loc[b, 'plan_cost'])))

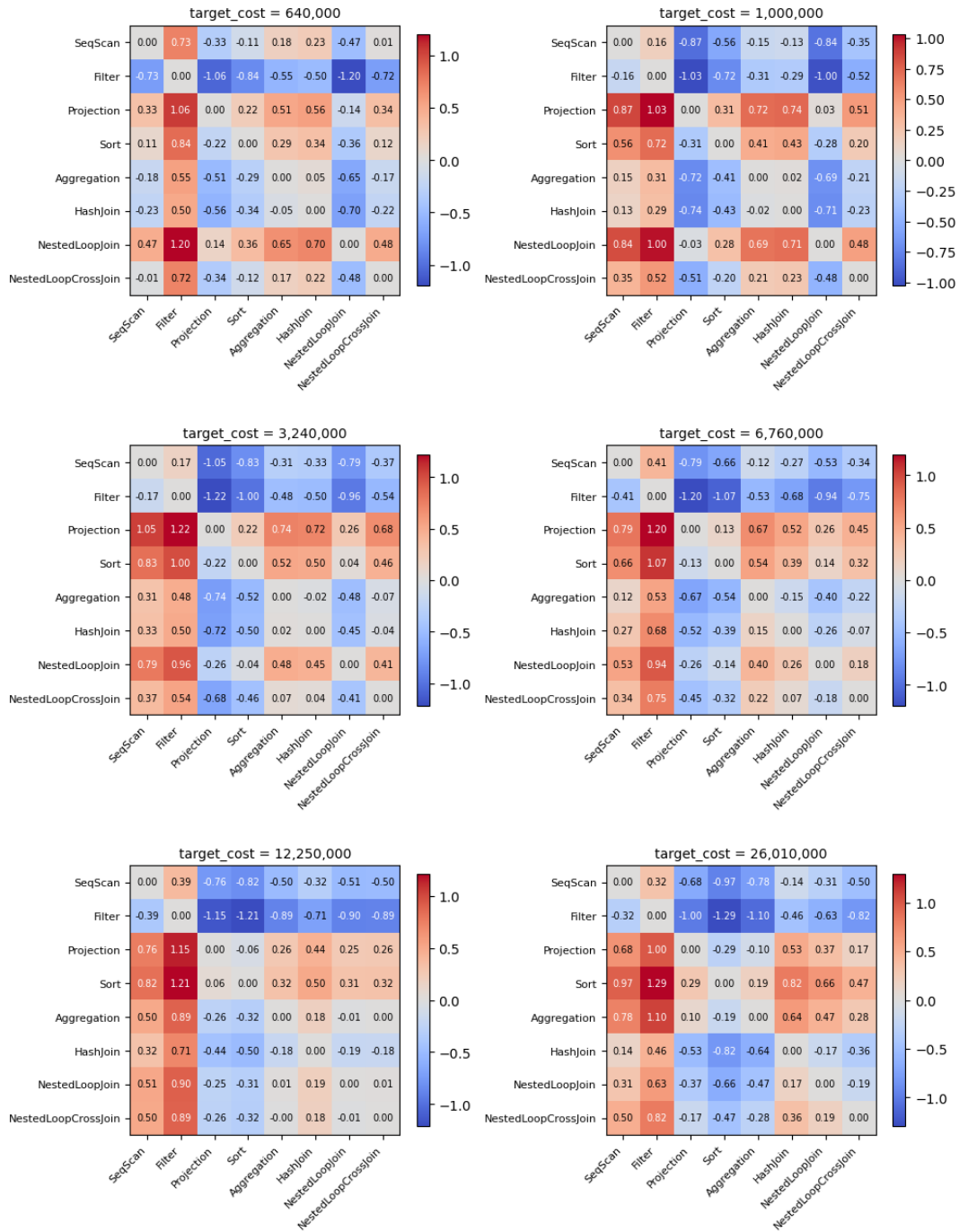
vmax = np.abs(mat).max() or 1.0
im = ax.imshow(mat, cmap='coolwarm', vmin=-vmax, vmax=vmax)
ax.set_xticks(range(m)); ax.set_xticklabels(ops, rotation=45, ha='right',
↪fontsize=8)
ax.set_yticks(range(m)); ax.set_yticklabels(ops, fontsize=8)
ax.set_title(f'target_cost = {target_cost:,}', fontsize=10)
plt.colorbar(im, ax=ax, shrink=0.8)
for i in range(m):
    for j in range(m):
        ax.text(j, i, f'{mat[i,j]:.2f}', ha='center', va='center',
↪fontsize=7,
                                color='black' if abs(mat[i,j]) < vmax * 0.6 else 'white')

for idx in range(len(matched_costs), nrows * ncols):
    axes[idx // ncols][idx % ncols].set_visible(False)

fig.suptitle('Local-cost-matched operators: log(cpu(a)/cpu(b)) -
↪log(plan_cost(a)/plan_cost(b)) [0 = perfect, +/-ln2 about 0.69 = 2x off]',
↪fontsize=12)
plt.tight_layout()
plt.show()

```

Local-cost-matched operators: $\log(\text{cpu}(a)/\text{cpu}(b)) - \log(\text{plan_cost}(a)/\text{plan_cost}(b))$ [0 = perfect, +/-ln2 about 0.69 = 2x off]



```
[13]: matched_rows = []
for target_cost in matched_costs:
    sub = matched_pivot.xs(target_cost, level='target_cost')
    ops = sub.index.tolist()
```

```

for a in ops:
    for b in ops:
        if a != b:
            delta = (np.log(sub.loc[a, 'cpu_time']) - np.log(sub.loc[b, 'cpu_time'])) -
                    (np.log(sub.loc[a, 'plan_cost']) - np.log(sub.loc[b, 'plan_cost']))
            matched_rows.append({'target_cost': target_cost, 'a': a, 'b': b, 'delta': delta})

matched_comparison = pd.DataFrame(matched_rows)
matched_comparison.pivot_table(index=['a', 'b'], columns='target_cost', values='delta').round(3)

```

```

[13]: target_cost
a      b      640000      1000000      3240000  \
Aggregation  Filter      0.552      0.312      0.478
           HashJoin      0.047      0.021     -0.025
           NestedLoopCrossJoin -0.169     -0.206     -0.065
           NestedLoopJoin     -0.651     -0.688     -0.479
           Projection     -0.508     -0.718     -0.743
           SeqScan      -0.180      0.149      0.307
           Sort      -0.290     -0.409     -0.523
Filter      Aggregation     -0.552     -0.312     -0.478
           HashJoin     -0.504     -0.290     -0.502
           NestedLoopCrossJoin -0.721     -0.517     -0.543
           NestedLoopJoin     -1.203     -0.999     -0.956
           Projection     -1.059     -1.030     -1.220
           SeqScan      -0.732     -0.162     -0.171
           Sort      -0.841     -0.720     -1.000
HashJoin    Aggregation     -0.047     -0.021      0.025
           Filter       0.504      0.290      0.502
           NestedLoopCrossJoin -0.217     -0.227     -0.041
           NestedLoopJoin     -0.698     -0.709     -0.454
           Projection     -0.555     -0.740     -0.718
           SeqScan      -0.227      0.128      0.331
           Sort      -0.337     -0.430     -0.498
NestedLoopCrossJoin  Aggregation      0.169      0.206      0.065
           Filter       0.721      0.517      0.543
           HashJoin      0.217      0.227      0.041
           NestedLoopJoin     -0.482     -0.482     -0.413
           Projection     -0.338     -0.512     -0.677
           SeqScan      -0.011      0.355      0.372
           Sort      -0.121     -0.203     -0.457
NestedLoopJoin  Aggregation      0.651      0.688      0.479
           Filter       1.203      0.999      0.956
           HashJoin      0.698      0.709      0.454

```

Projection	NestedLoopCrossJoin	0.482	0.482	0.413
	Projection	0.143	-0.030	-0.264
	SeqScan	0.471	0.837	0.786
	Sort	0.361	0.279	-0.044
	Aggregation	0.508	0.718	0.743
	Filter	1.059	1.030	1.220
	HashJoin	0.555	0.740	0.718
	NestedLoopCrossJoin	0.338	0.512	0.677
	NestedLoopJoin	-0.143	0.030	0.264
	SeqScan	0.328	0.867	1.049
SeqScan	Sort	0.218	0.309	0.220
	Aggregation	0.180	-0.149	-0.307
	Filter	0.732	0.162	0.171
	HashJoin	0.227	-0.128	-0.331
	NestedLoopCrossJoin	0.011	-0.355	-0.372
	NestedLoopJoin	-0.471	-0.837	-0.786
	Projection	-0.328	-0.867	-1.049
	Sort	-0.110	-0.558	-0.830
	Aggregation	0.290	0.409	0.523
	Filter	0.841	0.720	1.000
Sort	HashJoin	0.337	0.430	0.498
	NestedLoopCrossJoin	0.121	0.203	0.457
	NestedLoopJoin	-0.361	-0.279	0.044
	Projection	-0.218	-0.309	-0.220
	SeqScan	0.110	0.558	0.830
target_cost		6760000	12250000	26010000
a	b			
Aggregation	Filter	0.531	0.888	1.101
	HashJoin	-0.149	0.181	0.637
	NestedLoopCrossJoin	-0.220	0.000	0.277
	NestedLoopJoin	-0.405	-0.011	0.469
	Projection	-0.668	-0.262	0.103
	SeqScan	0.122	0.501	0.778
	Sort	-0.541	-0.320	-0.188
	Aggregation	-0.531	-0.888	-1.101
	HashJoin	-0.680	-0.707	-0.465
	NestedLoopCrossJoin	-0.751	-0.887	-0.824
Filter	NestedLoopJoin	-0.936	-0.899	-0.633
	Projection	-1.199	-1.150	-0.998
	SeqScan	-0.409	-0.386	-0.323
	Sort	-1.072	-1.207	-1.289
	Aggregation	0.149	-0.181	-0.637
	Filter	0.680	0.707	0.465
	NestedLoopCrossJoin	-0.071	-0.180	-0.359
	NestedLoopJoin	-0.256	-0.192	-0.168
	Projection	-0.519	-0.443	-0.533
HashJoin				

NestedLoopCrossJoin	SeqScan	0.271	0.321	0.142
	Sort	-0.392	-0.500	-0.825
	Aggregation	0.220	-0.000	-0.277
	Filter	0.751	0.887	0.824
	HashJoin	0.071	0.180	0.359
	NestedLoopJoin	-0.185	-0.011	0.191
	Projection	-0.448	-0.262	-0.174
NestedLoopJoin	SeqScan	0.342	0.501	0.501
	Sort	-0.321	-0.320	-0.465
	Aggregation	0.405	0.011	-0.469
	Filter	0.936	0.899	0.633
	HashJoin	0.256	0.192	0.168
	NestedLoopCrossJoin	0.185	0.011	-0.191
	Projection	-0.263	-0.251	-0.365
Projection	SeqScan	0.527	0.512	0.310
	Sort	-0.136	-0.309	-0.657
	Aggregation	0.668	0.262	-0.103
	Filter	1.199	1.150	0.998
	HashJoin	0.519	0.443	0.533
	NestedLoopCrossJoin	0.448	0.262	0.174
	NestedLoopJoin	0.263	0.251	0.365
SeqScan	SeqScan	0.790	0.763	0.675
	Sort	0.127	-0.058	-0.291
	Aggregation	-0.122	-0.501	-0.778
	Filter	0.409	0.386	0.323
	HashJoin	-0.271	-0.321	-0.142
	NestedLoopCrossJoin	-0.342	-0.501	-0.501
	NestedLoopJoin	-0.527	-0.512	-0.310
Sort	Projection	-0.790	-0.763	-0.675
	Sort	-0.663	-0.821	-0.966
	Aggregation	0.541	0.320	0.188
	Filter	1.072	1.207	1.289
	HashJoin	0.392	0.500	0.825
	NestedLoopCrossJoin	0.321	0.320	0.465
	NestedLoopJoin	0.136	0.309	0.657
	Projection	-0.127	0.058	0.291
	SeqScan	0.663	0.821	0.966